

Software Requirements Specification (SRS)

COVID-19 Cough Detection Web Application

Version 1.0

Prepared by: Drew Quashie, Amanda Ogbonna, Loleyi Oluwatomisin, Richard Alonso Garcia

Faculty Advisor: Dr. Zahra Nematzadeh

1 Introduction

1.1 Purpose

The purpose of this document is to define the requirements for a web-based application that pre-screens COVID-19 risk from recorded cough audio using machine learning. The SRS provides a shared understanding between the development team, the faculty advisor, and potential end-users. It specifies functional and non-functional requirements, external interfaces, constraints, and system characteristics in sufficient detail to guide design, development, and testing.

This application is intended to empower individuals by giving them an accessible tool for preliminary health monitoring. While the app will not replace professional medical testing or consultation, it serves as a triage aid, encouraging users who screen low to self-monitor, and users who screen high to seek testing sooner. This aligns with broader public health objectives of reducing avoidable visits to testing centers, easing the load on essential clinical workers, and improving early awareness of infection status.

COVID-19 remains a recurring public health concern, and during surges the majority of people with mild or uncertain symptoms cannot be tested immediately. Barriers such as clinic capacity, cost, and wait times delay confirmation and appropriate behavior (masking, isolation, care-seeking). By offering an easily accessible browser-based screen from nothing more than a microphone, this project addresses a pressing healthcare need while also contributing to the academic study of AI in healthcare.

1.2 Scope

The product is a cross-platform web application developed with React and FastAPI, powered by a PyTorch CNN trained on the CoughVid cough-audio dataset and gated by a pretrained XGBoost cough-presence classifier. The application will:

- Record a guided session of **three coughs** from the user's microphone in a single capture, with on-screen cues confirming each cough as it is detected.

- Validate the recording server-side, format, duration, SNR, clipping, and run a cough-presence gate so that non-cough audio is rejected before any analysis.
- Perform server-side AI analysis producing a three-class output (Negative / Uncertain / Positive) per cough via a 3-logit sigmoid CNN, aggregated into one final class by a deterministic majority vote.
- Display the risk level (Low / Medium / High), confidence percentage, vote breakdown, and per-cough signal strengths so results are transparent and auditable.
- Present medical disclaimers and risk-based recommendations (self-monitor, test, contact a provider) to promote responsible usage.
- Keep all health data on the user's device: analysis history and trends are stored in the browser's IndexedDB, and the server retains nothing after inference.

The app will not:

- Provide formal medical diagnoses. All results are pre-screening signals, always accompanied by disclaimers.
- Store user audio, features, or results on the server after an analysis completes.
- Replace official COVID-19 testing or professional medical evaluation, which remain essential for treatment and public health decisions.
- Perform analysis offline. History and trends are available offline, but active analysis requires a connection to the inference server (in-browser inference is a future enhancement).

1.3 Definitions, Acronyms, and Abbreviations

- **CNN**: Convolutional Neural Network, a deep learning model particularly effective for pattern recognition in spectrogram-like inputs.
- **MFCC**: Mel-Frequency Cepstral Coefficients, compressed cepstral features summarizing timbre and vocal-tract shape of an audio segment.
- **Mel Spectrogram**: Log-scaled time-frequency energy representation of audio.
- **SNR**: Signal-to-Noise Ratio, measure of cough signal strength against background noise.
- **CoughVid**: Public cough-audio dataset with status labels and symptom metadata, used for training.
- **XGBoost**: Gradient-boosted decision tree framework; here, the pretrained cough/no-cough gate.
- **ONNX**: Open Neural Network Exchange, portable model format for serving and future browser inference.
- **FastAPI**: Python web framework hosting the inference endpoint.
- **IndexedDB**: Browser-local database used for history, trends, and preferences.
- **Risk Level**: Categorized pre-screening assessment (Low / Medium / High) derived from model output.
- **Confidence Score**: Mean sigmoid signal strength of the winning class across the 3 coughs.
- **Cough Event**: One detected cough within the recording (three expected per session).
- **Segment**: Fixed-length (1 s) excerpt extracted from one cough event.
- **Vote**: Per-cough class decision aggregated into the final class by majority rule.

- **Inconclusive:** Result withheld because input quality, cough presence, or model confidence is below threshold.

1.4 References

- IEEE Std 830-1998 (Software Requirements Specifications Recommended Practice).
- CoughVid Dataset Documentation, ETH Zurich.
- Team Project Plan and Presentation documents.
- Research publications on environmental sound classification with two-stream CNNs and multi-feature fusion (see SDD §9).

1.5 Overview

The remainder of this document is structured as follows:

- Section 2 provides an overall description of the system.
- Section 3 specifies detailed functional and non-functional requirements, followed by use case scenarios.
- Section 4 includes supporting information such as appendices, references, and risk assessments.

2 Overall Description

2.1 Product Perspective

The system is a standalone web application built as a three-tier architecture: a React presentation layer in the browser, a FastAPI application-logic layer, and a PyTorch machine-learning layer, with browser-side persistence in IndexedDB. Unlike many health applications that require clinic visits or specialized hardware, this app needs nothing beyond a standard device microphone and a modern browser.

Server-side inference was chosen over client-side execution for v1: a full PyTorch training and experimentation pipeline is already in place, and server-side models can be updated without redeploying the client. The cost, an upload per analysis, is mitigated by keeping clips small (one recording, 5–15 s, under 1 MB) and the pipeline fast (quality gate fails bad inputs before feature extraction). The ML layer is deliberately kept free of FastAPI dependencies so the trained model can later be exported to ONNX and run directly in the browser (ONNX Runtime Web), which would restore fully offline analysis. This approach follows the broader trend in healthcare technology of progressively moving lightweight models closer to, and eventually onto, the user's device.

Privacy is enforced architecturally rather than by policy: the server is stateless with respect to health data. An inference request goes in, a result comes out, and neither the audio, the features, nor the result is retained.

2.2 Product Functions

The web application provides several core functions that together enable its role as a pre-screening tool. Each function balances usability, technical feasibility, and ethical responsibility.

- **Guided three-cough capture with live level meter.** The user opens the recorder, grants microphone permission, and follows a prompt to give three loud, clear coughs, one at a time. On-screen cue dots (○ ○ ○) fill as each cough is detected in real time, and capture stops automatically after the third. A live input-level meter confirms the microphone is hearing the user before they commit to a session. This standardization reduces rejected submissions and makes downstream segmentation deterministic, the system always expects three cough events.
- **Server-side validation, cough gating, and preprocessing.** Each uploaded recording is first checked for format, duration bounds, SNR, and clipping. A pretrained XGBoost cough/no-cough classifier (from the CoughVid repository) then confirms the clip actually contains cough audio; non-cough input is rejected with an explanatory Inconclusive result before any model work occurs. The segmenter locates the three cough events and cuts one fixed 1-second window per event, which are resampled, normalized, and converted to joint MFCC + log-Mel features.
- **CNN classification with confidence and vote display.** At the core of the application is a PyTorch CNN (ResNet-style transfer learning) with a 3-logit sigmoid output head trained on CoughVid. Each of the three cough segments produces an independent per-cough class, and the three are combined by a deterministic majority vote (with a mean-sigmoid tie-break for a 1-1-1 split). The result screen shows the risk level, a confidence percentage, the full vote breakdown, and the per-class signal strengths, so users and reviewers can see exactly how the conclusion was reached.
- **Disclaimers and risk-based recommendations.** Every prediction is displayed alongside a mandatory disclaimer that the app is not a diagnostic tool, plus recommendations matched to the risk level: continued monitoring for Low, testing and masking for Medium, prompt testing and provider contact for High. This ensures the app never exists in isolation, it directs users toward official testing and appropriate care.
- **Longitudinal trends dashboard with export.** Every saved analysis is written to IndexedDB on the user's device. The trends screen plots risk level and confidence over time and summarizes activity, giving users a view of their condition across days. History can be exported as CSV or JSON so the record can be shared with a clinician, the mechanism by which medical professionals receive patient progress without any server-side storage or accounts in v1.

2.3 User Characteristics

The application is intended for use by non-specialist individuals seeking a quick, low-effort pre-screen, plus the clinicians they may share results with. The user base is profiled along three characteristics:

- **Demographics:** Adults concerned about possible COVID-19 exposure or symptoms, across a wide range of ages and technical environments, at home, at work, or on the go,

who have a laptop or phone with a working microphone. Secondly, medical professionals receive exported trend records from patients and use them to gauge progression before or between visits.

- **Technical skills:** The app assumes only basic browser literacy: opening a website, allowing microphone access when prompted, clicking a record button, and reading a result screen. No familiarity with machine learning, audio engineering, or configuration is expected. The entire capture flow is one screen and one button.
- **Motivation:** Users are primarily motivated by reassurance, risk awareness, and avoiding unnecessary trips to testing centers. Some will use the app proactively after a known exposure; others when feeling symptoms. The app provides peace of mind when results are low-risk and a clear, actionable nudge toward testing when they are not.

2.4 Constraints

- **Hardware:** The application must run on ordinary consumer devices, laptops, tablets, and phones with built-in microphones, with no specialized recording hardware. This constrains input quality: microphones vary widely in frequency response and noise floor, so the pipeline must tolerate noisy, clipped, or uneven recordings by rejecting them gracefully rather than producing unreliable results.
- **Regulatory and ethical:** Because the application touches health-related predictions, it must comply with healthcare app ethics standards, informed consent norms, and transparency requirements. Predictions must never be presented as official diagnoses or substitutes for testing; doing so could create liability and mislead users. All results must be framed as pre-screening, accompanied by disclaimers urging confirmation through official testing channels.
- **Privacy:** All persistent health data must remain on the user's device (IndexedDB). The server must not retain audio, derived features, or results after inference completes, and server logs must not contain health-related content. This design aligns with the strict-control expectations of frameworks such as HIPAA and GDPR for personally identifiable health data, and it is enforced by architecture (stateless inference) rather than by promise.
- **Usability:** The interface must be usable across modern browsers and screen sizes, from phone-width to desktop, with a responsive single-page layout. Risk levels must be conveyed by color and text together (never color alone), and the design must remain intuitive for first-time users, minimizing the steps from landing page to result.
- **Performance:** End-to-end latency and payload size are constrained by the SDD §8.1 targets (capture-to-result under 6 s, server inference under 1.5 s including the cough gate and three forward passes, uploads under 1 MB). These are binding: long delays erode the app's value as a quick screen.

2.5 Assumptions and Dependencies

- **Users will have a working microphone, a modern browser, and basic technical skills.** The application depends on `getUserMedia`/Web Audio API support and the user granting microphone permission. Basic digital literacy (opening a site, clicking a button) is assumed.

- **The CoughVid dataset and its pretrained XGBoost cough classifier will remain available.** Training depends on CoughVid's continued availability, and the cough-presence gate depends on the detector artifact distributed with its repository. If either becomes unavailable, retraining and gating must be re-sourced.
- **PyTorch, FastAPI, and React will continue to be supported.** The deployment strategy relies on these frameworks; their deprecation would force migration and potential architectural changes.
- **A reasonably quiet recording environment will be available.** Accurate analysis depends on audible, unclobbered coughs. The system detects and rejects poor conditions (low SNR, clipping), but the assumption of basic recordability remains, the quality gate is a safety net, not a substitute for signal.
- **Users will accept disclaimers acknowledging the app is not a diagnostic tool.** Clear disclaimers are presented with every result; users are expected to read and accept them, protecting both users (from misunderstanding) and the team (from liability).

2.6 Apportioning of Requirements

Future versions of the application may include features beyond the initial release:

- **In-browser inference** via ONNX Runtime Web, removing the upload round-trip, enabling offline analysis, and eliminating server-side processing entirely.
- **Account-based sharing** so clinicians can view a patient's longitudinal dashboard remotely instead of receiving manual CSV/JSON exports.
- **Multi-language support** to broaden accessibility across regions with high COVID-19 prevalence.
- **Richer longitudinal analytics**, symptom overlay, cough-frequency tracking per the original project vision, and trend comparisons across users (anonymously, opt-in).
- **Additional respiratory screens** (influenza, RSV) reusing the same capture and inference pipeline with different model heads.

3 Specific Requirements

3.1 Software Functional Requirements (SFRs)

Each SFR includes rationale and, where possible, an example use.

SFR1: Request and handle microphone permission and record audio via the Web Audio API. Rationale: Foundation of the capture flow; failure here blocks all analysis. Example: A user opens the site, clicks record, and grants microphone access in the browser prompt.

SFR2: Record one session of three guided coughs. Rationale: Standardized input makes segmentation deterministic (exactly three cough events expected) and improves vote aggregation. Example: The recorder shows "cough 3 times"; cue dots fill ● ● ○ as coughs are detected; capture auto-stops after the third (manual stop also allowed).

SFR3: Display a live input level meter and run client-side sanity checks. Rationale: Immediate feedback prevents uploading unusable recordings. Example: If the meter stays flat, the user sees a permissions/no-signal hint before wasting a take; gross clipping or pure silence triggers an instant recapture prompt.

SFR4: Upload the single recording to `POST /v1/analyze`. Rationale: One request per analysis keeps the API contract simple and the round-trip count at one. Example: A 5–15 s WEBM/WAV clip under 1 MB is sent as multipart form data.

SFR5: Validate input server-side (format, duration, size) and run the SNR/clipping quality gate. Rationale: Bad inputs fail fast, before any model work, with an explanatory result. Example: A 0.3 s clip returns Inconclusive("duration too short") instead of an error page.

SFR6: Run the cough-presence gate (pretrained XGBoost classifier) before segmentation. Rationale: Rejects non-cough audio (music, speech, ambient noise) at the earliest server stage. Example: Uploading a hummed tune returns Inconclusive("cough not detected") and no segmentation is attempted.

SFR7: Segment exactly three onset-centered 1 s windows. Rationale: Fixed-length input matches training conditions; onset-centering captures the full cough event. Example: Each detected cough is cut to a 1.0 s window (the rounded 98.5th-percentile CoughVid cough length); shorter events are padded with silence.

SFR8: Extract joint MFCC + log-Mel features per segment with training-set normalization. Rationale: The dual feature representation is the model's documented input contract (timbre + spectral detail). Example: Each segment yields a stacked feature map normalized by the training mean/std.

SFR9: Run CNN inference three times; each output is 3 logits with sigmoid activation; per-cough class = argmax. Rationale: Delivers the model's core classification with an explicit, inspectable per-cough decision. Example: Cough 1 sigmoids [0.12, 0.09, 0.81] → class Positive.

SFR10: Aggregate the three per-cough classes by deterministic majority vote. Rationale: Identical inputs must yield identical outputs; voting suppresses single-cough noise. Example: Votes [Positive, Positive, Negative] → Positive; a 1-1-1 split falls back to argmax of the mean sigmoid vector.

SFR11: Enforce a confidence gate and display confidence as a percentage. Rationale: Withheld results are more honest than low-certainty guesses; percentage conveys certainty transparently. Example: If the mean sigmoid of the winning class across the three coughs is below 0.50, the result is Inconclusive("low model confidence").

SFR12: Display the result: risk level, confidence, vote breakdown, per-cough classes, signal strengths, and quality line. Rationale: Full transparency lets users and reviewers audit how the

conclusion was reached. Example: Result screen shows "MEDIUM RISK · Confidence 62% · Vote: 2 of 3 → POSITIVE · Signal: Neg 0.31 / Unc 0.18 / Pos 0.62".

SFR13: Present disclaimers and risk-based recommendations on every result. Rationale: Ethical compliance and correct interpretation. Example: Every result carries "Pre-screening tool only, not a diagnosis," plus the recommendation text matched to Low/Medium/High.

SFR14: Save analysis records to IndexedDB and manage history (list, delete-all). Rationale: Local persistence enables longitudinal tracking with zero server storage. Example: After saving, the dashboard lists "Sep 24 · LOW · 78%"; the user can clear all data from settings.

SFR15: Render a longitudinal trends dashboard with CSV/JSON export. Rationale: Core novel feature, condition-over-time visibility for patients and (via export) clinicians. Example: The trends screen plots risk and confidence over 30 days and offers [Export CSV] / [Export JSON].

SFR16: Provide explicit UI states (idle, recording, analyzing, result, inconclusive) in a responsive layout. Rationale: Setting expectations at each step prevents perceived hangs and confusion. Example: Uploading shows a spinner labeled "Analyzing..."; the inconclusive state offers [Try Again].

3.2 Software Non-Functional Requirements (SNFRs)

SNFR1: Capture-to-result latency under 6 s end-to-end; server inference under 1.5 s.

Explanation: Long delays reduce usability as a quick screen; the 1.5 s budget covers the XGBoost gate plus three forward passes.

SNFR2: Server processing is stateless, no audio, features, or results retained after inference. Explanation: Privacy is enforced architecturally; nothing sensitive exists to breach.

SNFR3: Handle corrupted, irrelevant, or degraded inputs gracefully. Example: Non-cough audio, corrupted files, too-short/too-long clips, silence, and heavy clipping all return a specific Inconclusive reason without crashing or returning misleading results.

SNFR4: Interface intuitive for first-time users. Explanation: The full flow is one screen and one button; usability testing will confirm no tutorial is needed.

SNFR5: Runs on modern desktop and mobile browsers (Chrome, Edge, Firefox, Safari).

Explanation: Web Audio API, IndexedDB, and ES2020+ baseline ensure broad adoption without per-browser builds.

SNFR6: No health-related data in server logs. Explanation: Logs carry timings, trace IDs, and error classes, never audio content, results, or identifiers that reveal health status.

SNFR7: Accessible, risk shown by color AND text; large-font/high-contrast support.

Explanation: Color-only status fails for colorblind users; the responsive design must scale with browser font settings.

SNFR8: Modular layered architecture; the ML layer has no FastAPI dependencies.

Explanation: Facilitates maintenance, isolated testing, and the future browser-inference migration.

SNFR9: Models and datasets are swappable. Explanation: The retraining pipeline (CoughVid → features → CNN → ONNX) must accept updated datasets or architectures without changes to the application layers.

SNFR10: Healthcare ethics compliance, disclaimers, pre-screening framing, transparent confidence. Explanation: Aligns with regulatory and institutional expectations for health-adjacent applications; no result may be presented as a diagnosis.

3.3 Use Case Scenarios (UCSs)

- **UCS1: Normal Three-Cough Analysis**, Precondition: User on the site, microphone permission granted. Steps: Open recorder → Start recording → Give 3 coughs (cue dots fill) → Upload automatic → Server validates, gates, segments ×3, classifies ×3, votes → Result displayed with risk level, confidence, vote breakdown, and disclaimer. Postcondition: User receives an advisory risk rating; result saved locally if they choose [Save Result].
- **UCS2: Non-Cough or Poor-Quality Recording**, Precondition: User on the recorder. Steps: Start recording → Speak, hum, or record in heavy noise → Upload → Server quality gate or XGBoost cough gate fails → Inconclusive result with specific reason ("cough not detected" / "low SNR") and [Try Again]. Postcondition: No misleading result produced; user recaptures.
- **UCS3: High-Risk Result**, Precondition: User submits three coughs that vote Positive with mean Positive signal ≥ 0.65 . Steps: Analysis completes → HIGH risk displayed with confidence and vote breakdown → Recommendations: take an official test soon, contact a provider if symptomatic, disclaimer shown. Postcondition: User is directed toward official confirmation and appropriate care.
- **UCS4: Review Trends and Export History**, Precondition: At least one saved analysis exists. Steps: Open Trends → View risk/confidence over time and activity summary → Select Export CSV or Export JSON → File downloads → User shares it with a clinician. Postcondition: Clinician receives the longitudinal record without any server-side account or storage.

4 Supporting Information

4.1 Table of Contents and Index

The table of contents and index are navigation aids. The TOC lists sections and subsections; the index covers technical terms, acronyms, and requirement identifiers (SFR/SNFR/UCS).

4.2 Appendices

Appendices supplement requirements with supporting information:

- **Appendix A: Milestones and Deliverables**, Expanded timeline and evaluation points (Milestone 1: Sep 28, planning, dataset pipeline, augmentation, baseline model + recorder component; Milestone 2: Oct 26, model training/evaluation, FastAPI endpoints, initial UI; Milestone 3: Nov 23, end-to-end integration, trends dashboard, testing, final deliverables).
- **Appendix B: Task Matrix**, Clear breakdown of responsibilities: Requirements & project plan (Amanda Ogbonna); preprocessing + augmentation pipeline (Drew Quashie); baseline CNN architecture (Loleyi Oluwatomisin, Amanda Ogbonna); React recording interface (Richard Alonso Garcia).
- **Appendix C: Risk Assessment**, Dataset class imbalance and demographic generalization; the third-class label engineering problem (CoughVid status labels are binary; "Uncertain" is derived from self-reported symptom metadata, flagged for advisor review); ambient noise and microphone variability; end-to-end latency against the 6 s target; ethical risk of perceived diagnosis (mitigated by mandatory disclaimers and pre-screening framing).
- **Appendix D: Future Enhancements**, In-browser ONNX inference, clinician accounts for remote trend sharing, multi-language support, additional respiratory model heads.

4.3 References to Supporting Documents

- IEEE Std 830-1998.
- CoughVid Dataset Documentation.
- Team project plan, software design document, and presentation materials.